Seminar 2 - Backend cu Node.js, Express, Prisma si JWT

Cuprins

1. Node.js si npm
2. Express: primul server
3. Middleware si tratarea erorilor
4. Prisma ORM si baza de date
5. CRUD pentru produse
6. Autentificare cu JWT
7. Autorizare: middleware si roluri
8. Relatii si comenzi
9. Structura finala si checklist

Exemplele din seminar continua contextul de **magazin online** din Seminar 1. La finalul seminarului vom avea un **API REST** complet pentru aplicatia e-commerce, pe care il vom consuma din React in Seminarul 3.

Cerinte preliminare

- Node.js versiunea 20 sau mai noua (`node -v`)
- un editor de cod (VS Code)
- Postman, Thunder Client sau `curl` pentru testarea API-ului

1. Node.js si npm

Node.js este un mediu de executie care permite rularea JavaScript in afara browserului, de exemplu pe un server.

npm (Node Package Manager) instaleaza si gestioneaza bibliotecile externe (pachetele).

Verificare:

```
node -v
npm -v
```

Crearea proiectului

```
mkdir ecommerce-api
cd ecommerce-api

npm init -y
```

Comanda creeaza fisierul `package.json`, care descrie proiectul si dependentele lui.

Module ES6

In Seminar 1 am folosit `import` / `export`. In Node.js le activam din `package.json`:

```
{
    "name": "ecommerce-api",
    "version": "1.0.0",
    "type": "module",
    "scripts": {
        "dev": "node --watch src/index.js",
        "start": "node src/index.js"
    }
}
```

- `"type": "module"` activeaza sintaxa `import` / `export`;
- `node --watch` reporneste automat serverul cand modificam un fisier.

Dependente

```
npm install express cors dotenv bcryptjs jsonwebtoken cookie-parser
npm install @prisma/client@6
npm install -D prisma@6
```

Pachet	Rol
`express`	framework web pentru API
`cors`	permite request-uri din browser, de pe alt domeniu/port
`dotenv`	incarca variabilele din `.env`
`bcryptjs`	criptarea (hash-ul) parolelor
`jsonwebtoken`	generarea si verificarea token-urilor JWT
`cookie-parser`	citeste cookie-urile din request si le pune in `req.cookies`
`prisma`, `@prisma/client`	ORM pentru baza de date

> Folosim Prisma 6 (`@6`) pentru ca setup-ul din seminar sa fie identic pentru toti. Versiunile mai noi au o configurare diferita.

Variabile de mediu

Secretele si configurarile nu se scriu direct in cod. Le punem in fisierul `.env`:

```
PORT=3000
JWT_SECRET=schimba-ma-cu-un-secret-lung-si-aleatoriu
JWT_EXPIRES_IN=1h
NODE_ENV=development
```

Fisierul `.gitignore`:

```
node_modules
.env
prisma/dev.db
```

> `.env` **nu se urca niciodata pe Git**. Pentru colegi se poate adauga un fisier `.env.example` fara valorile reale.

Accesare in cod:

```
const port = process.env.PORT ?? 3000;
```

Aici folosim `??` din Seminar 1: daca `PORT` nu exista, se foloseste `3000`.

Structura proiectului

```
ecommerce-api/
├── prisma/
│   ├── schema.prisma
│   └── seed.js
├── src/
│   ├── index.js
│   ├── lib/
│   │   └── prisma.js
│   ├── middleware/
│   │   ├── auth.js
│   │   └── errorHandler.js
│   ├── routes/
│   │   ├── auth.routes.js
│   │   ├── products.routes.js
│   │   └── orders.routes.js
│   └── utils/
│       ├── httpError.js
│       └── cookies.js
├── .env
├── .gitignore
└── package.json
```

Cream folderele:

```
mkdir -p src/lib src/middleware src/routes src/utils
```

2. Express: primul server

HTTP pe scurt

Frontend-ul trimite **request-uri** catre backend, iar backend-ul raspunde cu **response-uri**, de obicei in format JSON (Seminar 1, sectiunea 8).

Metoda	Rol	Exemplu
`GET`	citeste date	`GET /api/products`
`POST`	creeaza o resursa	`POST /api/products`
`PATCH`	modifica partial o resursa	`PATCH /api/products/3`
`PUT`	inlocuieste o resursa	`PUT /api/products/3`
`DELETE`	sterge o resursa	`DELETE /api/products/3`

Coduri de status folosite frecvent:

Cod	Semnificatie
`200`	OK
`201`	Creat
`204`	OK, fara continut (stergere)
`400`	Request invalid (date gresite)
`401`	Neautentificat
`403`	Autentificat, dar fara drepturi
`404`	Resursa nu exista
`409`	Conflict (ex.: email deja folosit)
`500`	Eroare interna de server

Primul server

Fisierul `src/index.js`:

```
import "dotenv/config";
import express from "express";
import cors from "cors";

const app = express();
const PORT = process.env.PORT ?? 3000;

app.use(cors({ origin: "http://localhost:5173" }));
```

```
app.use(express.json());

app.get("/api/health", (req, res) => {
    res.json({ status: "UP" });
});

app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

Pornire:

```
npm run dev
```

Test:

```
curl http://localhost:3000/api/health
```

Rezultat:

```
{ "status": "ok" }
```

- `express.json()` citeste corpul request-ului JSON si il pune in `req.body`;
- `cors(...)` permite request-uri de la aplicatia React (Vite ruleaza implicit pe portul `5173`).

Request si Response

```
app.get("/api/demo/:id", (req, res) => {

    console.log(req.params);   // { id: "5" }          -> din URL
    console.log(req.query);    // { category: "..." }  -> dupa "?"
    console.log(req.body);     // corpul JSON (la POST/PATCH)
    console.log(req.headers);  // header-ele request-ului

    res.status(200).json({ message: "ok" });
});
```

Sursa	Exemplu URL	Acces
Parametru de ruta	`/api/products/5`	`req.params.id`
Query string	`/api/products?category=Electronics`	`req.query.category`
Body	`POST` cu JSON	`req.body`

> `req.params.id` este intotdeauna **string**. Il convertim cu `Number(...)` cand avem nevoie de numar.

Exemplu cu date in memorie

Refolosim lista din Seminar 1, temporar, inainte sa folosim o baza de date:

```
const products = [
    { id: 1, name: "Laptop", price: 4000, category: "Electronics" },
    { id: 2, name: "Phone", price: 2500, category: "Electronics" },
    { id: 3, name: "Desk", price: 800, category: "Furniture" }
];

app.get("/api/demo/products", (req, res) => {
    const { category } = req.query;

    const result = category
        ? products.filter(product => product.category === category)
        : products;

    res.json(result);
});

app.get("/api/demo/products/:id", (req, res) => {
    const product = products.find(
        product => product.id === Number(req.params.id)
    );

    if (!product) {
        return res.status(404).json({ message: "Produsul nu exista" });
    }

    res.json(product);
});
```

Test:

```
curl "http://localhost:3000/api/demo/products?category=Furniture"
```

> Acest cod demonstrativ se sterge dupa ce trecem la Prisma.

Router

Pentru organizare, rutele se grupeaza in fisiere separate, folosind `Router`:

```
import { Router } from "express";

const router = Router();
```

```
router.get("/", (req, res) => {
    res.json([]);
});

export default router;
```

Router-ul se monteaza in `index.js`:

```
import productRoutes from "./routes/products.routes.js";

app.use("/api/products", productRoutes);
```

Ruta `router.get("/")` devine astfel `GET /api/products`.

3. Middleware si tratarea erorilor

Un **middleware** este o functie care se executa intre primirea request-ului si trimiterea raspunsului.

```
Request
   ↓
middleware 1 (cors)
   ↓
middleware 2 (express.json)
   ↓
middleware 3 (authenticate)
   ↓
handler-ul rutei
   ↓
Response
```

Structura unui middleware:

```
// src/middleware/logger.js
const logger = (req, res, next) => {
    console.log(`${req.method} ${req.url}, body: ${JSON.stringify(req.body ||
{})}`);
    next();
}

export default logger;

// index.js
app.use(logger);
```

`next()` trece controlul la urmatorul pas. Daca nu este apelat si nu trimitem un raspuns, request-ul ramane blocat.

> **Ordinea conteaza.** Middleware-urile se executa in ordinea in care sunt inregistrate.

Clasa `HttpError`

Pentru a semnala erori cu un cod de status, cream o clasa proprie.

Fisierul `src/utils/httpError.js`:

```
export class HttpError extends Error {
    constructor(status, message) {
        super(message);
        this.status = status;
```

```
    }
}
```

Utilizare:

```
throw new HttpError(404, "Produsul nu exista");
```

Error handler global

Un middleware de erori are **4 parametri**: `(err, req, res, next)`.

Fisierul `src/middleware/errorHandler.js`:

```
const notFound = (req, res) => {
    res.status(404).json({
        message: `Ruta ${req.method} ${req.originalUrl} nu exista`
    });
}

const errorHandler = (err, req, res, next) => {

    // Erori Prisma frecvente
    if (err.code === "P2002") {
        return res.status(409).json({ message: "Valoarea exista deja" });
    }

    if (err.code === "P2025") {
        return res.status(404).json({ message: "Resursa nu a fost gasita" });
    }

    if (err.code === "P2003") {
        return res.status(409).json({
            message: "Resursa este folosita de alte inregistrari"
        });
    }

    const status = err.status ?? 500;

    if (status === 500) {
        console.error(err);
    }

    res.status(status).json({
        message: status === 500 ? "Eroare interna de server" : err.message
    });
}

export { notFound, errorHandler };
```

Se inregistreaza **la final**, dupa toate rutele:

```
import { notFound, errorHandler } from "./middleware/errorHandler.js";

// ... rutele ...

app.use(notFound);
app.use(errorHandler);
```

Erori in functii `async`

In Express 5 (versiunea instalata implicit acum), daca un handler `async` arunca o eroare sau un Promise este respins, eroarea ajunge **automat** in `errorHandler`:

```
router.get("/:id", async (req, res) => {
    const product = await prisma.product.findUnique({ ... });

    if (!product) {
        throw new HttpError(404, "Produsul nu exista");
    }

    res.json(product);
});
```

Nu mai este nevoie de `try` / `catch` in fiecare ruta.

> In Express 4 era necesar `try` / `catch` + `next(error)` in fiecare handler `async`.

Format uniform pentru erori:

```
{ "message": "Produsul nu exista" }
```

Frontend-ul va citi `error.response.data.message` (Seminar 3).

4. Prisma ORM si baza de date

Prisma este un ORM (Object-Relational Mapper). Descriem tabelele intr-un fisier `schema.prisma`, iar Prisma genereaza un client JavaScript pentru interogari.

In loc de SQL:

```
SELECT * FROM Product WHERE category = 'Electronics';
```

scriem:

```
await prisma.product.findMany({
    where: { category: "Electronics" }
});
```

In seminar folosim **SQLite** (o baza de date intr-un singur fisier, fara instalari). Schema este aceeasi si pentru PostgreSQL sau MySQL, cu mici diferente de configurare.

Initializare

```
npx prisma init --datasource-provider sqlite
```

Comanda creeaza:

- `prisma/schema.prisma`
- `.env` cu variabila `DATABASE_URL`

Verificati ca `.env` contine si variabilele din sectiunea 1:

```
DATABASE_URL="file:./dev.db"
PORT=3000
JWT_SECRET=schimba-ma-cu-un-secret-lung-si-aleatoriu
JWT_EXPIRES_IN=1h
NODE_ENV=development
```

Schema

Fisierul `prisma/schema.prisma`:

```
generator client {
    provider = "prisma-client-js"
```

```
}

datasource db {
    provider = "sqlite"
    url      = env("DATABASE_URL")
}

model User {
    id        Int      @id @default(autoincrement())
    email     String   @unique
    name      String
    password  String
    role      String   @default("USER")
    createdAt DateTime @default(now())
}

model Product {
    id          Int      @id @default(autoincrement())
    name        String
    description String   @default("")
    price       Float
    category    String
    stock       Int      @default(0)
    imageUrl    String?
    createdAt   DateTime @default(now())
}
```

Element	Semnificatie
`@id`	cheia primara
`@default(autoincrement())`	id generat automat
`@unique`	valoarea nu se poate repeta
`String?`	camp optional (poate fi `null`)
`@default(now())`	data curenta la creare

`role` poate avea valorile `"USER"` sau `"ADMIN"`.

Migrare

O **migrare** aplica schema in baza de date si pastreaza istoricul modificarilor.

```
npx prisma migrate dev --name init
```

Rezultat:

- se creeaza fisierul bazei de date `prisma/dev.db`;
- se creeaza folderul `prisma/migrations/`;

- se genereaza clientul Prisma.

De fiecare data cand modificam `schema.prisma`, rulam din nou `migrate dev` cu un nume sugestiv.

Clientul Prisma

Fisierul `src/lib/prisma.js`:

```
import { PrismaClient } from "@prisma/client";

export const prisma = new PrismaClient();
```

Cream **o singura instanta**, pe care o importam unde avem nevoie.

Prisma Studio

Interfata vizuala pentru baza de date:

```
npx prisma studio
```

Se deschide in browser, la `http://localhost:5555`.

Seed: date initiale

Fisierul `prisma/seed.js`:

```
import { PrismaClient } from "@prisma/client";
import bcrypt from "bcryptjs";

const prisma = new PrismaClient();

async function main() {

    await prisma.user.upsert({
        where: { email: "admin@shop.com" },
        update: {},
        create: {
            email: "admin@shop.com",
            name: "Admin",
            password: await bcrypt.hash("admin123", 10),
            role: "ADMIN"
        }
    });

    const products = [
        { name: "Laptop", price: 4000, category: "Electronics", stock: 10 },
        { name: "Phone", price: 2500, category: "Electronics", stock: 25 },
        { name: "Tablet", price: 1800, category: "Electronics", stock: 15 },
```

```
        { name: "Headphones", price: 350, category: "Electronics", stock: 40 },
        { name: "Desk", price: 800, category: "Furniture", stock: 5 },
        { name: "Chair", price: 500, category: "Furniture", stock: 12 },
        { name: "Bookshelf", price: 650, category: "Furniture", stock: 0 }
    ];

    await prisma.product.deleteMany();

    await prisma.product.createMany({
        data: products.map(product => ({
            ...product,
            description: `${product.name} - produs din categoria
${product.category}`,
            imageUrl: `https://picsum.photos/seed/${product.name}/400/300`
        }))
    });

    console.log("Seed finalizat");
}

main()
    .catch(error => {
        console.error(error);
        process.exit(1);
    })
    .finally(() => prisma.$disconnect());
```

Observati folosirea `map`, spread si template literals din Seminar 1.

In `package.json` adaugam:

```
"prisma": {
    "seed": "node prisma/seed.js"
}
```

Rulare:

```
npx prisma db seed
```

> Seed-ul sterge produsele existente (`deleteMany`). Nu il rulati dupa ce ati creat comenzi, pentru ca produsele sunt referite de acestea. Daca se intampla, resetati baza cu `npx prisma migrate reset`.

Operatii Prisma

Operatie	Rol
`findMany`	returneaza o lista

Operatie	Rol
`findUnique`	returneaza o inregistrare dupa camp unic (`id`, `email`) sau `null`
`create`	insereaza
`update`	modifica o inregistrare
`delete`	sterge o inregistrare
`count`	numara inregistrarile

Exemple:

```
// filtrare
await prisma.product.findMany({
    where: {
        category: "Electronics",
        price: { gte: 1000, lte: 5000 }
    },
    orderBy: { price: "asc" }
});

// cautare dupa text
await prisma.product.findMany({
    where: { name: { contains: "lap" } }
});

// doar anumite campuri
await prisma.product.findMany({
    select: { id: true, name: true, price: true }
});

// modificare
await prisma.product.update({
    where: { id: 1 },
    data: { price: 3799 }
});

// stoc: scadere atomica
await prisma.product.update({
    where: { id: 1 },
    data: { stock: { decrement: 2 } }
});
```

Operatorii de filtrare: `equals`, `not`, `in`, `lt`, `lte`, `gt`, `gte`, `contains`, `startsWith`.

5. CRUD pentru produse

CRUD = Create, Read, Update, Delete.

Conventii REST

Metoda	Ruta	Actiune
GET	/api/products	lista produselor
GET	/api/products/:id	un produs
DELETE	/api/products/:id	sterge produs
POST	/api/products	creeaza un produs

Functii utilitare

Fisierul src/utils/parseId.js:

```
import { HttpError } from "./httpError.js";

export const parseId = (value) => {
    const id = Number(value);

    if (!Number.isInteger(id) || id < 1) {
        throw new HttpError(400, "Id invalid");
    }

    return id;
}
```

Router-ul de produse

Fisierul src/routes/products.routes.js:

```
import { Router } from "express";
import { prisma } from "../lib/prisma.js";
import { HttpError } from "../utils/httpError.js";
import { parseId } from "../utils/parseId.js";

const router = Router();

// GET /api/products?category=...&search=...
router.get("/", async (req, res) => {
    const { category, search } = req.query;

    const products = await prisma.product.findMany({
```

```
        where: {
            ...(category && { category }),
            ...(search && { name: { contains: search } })
        },
        orderBy: { createdAt: "desc" }
    });

    res.json(products);
});

// GET /api/products/:id
router.get("/:id", async (req, res) => {
    const product = await prisma.product.findUnique({
        where: { id: parseId(req.params.id) }
    });

    if (!product) {
        throw new HttpError(404, "Produsul nu exista");
    }

    res.json(product);
});

export default router;
```

Observatii:

- `...(category && { category })` adauga filtrul doar daca parametrul exista;

Montarea in `index.js`

Versiunea completa a fisierului, pana in acest punct:

```
import "dotenv/config";
import express from "express";
import cors from "cors";

import productRoutes from "./routes/products.routes.js";
import { notFound, errorHandler } from "./middleware/errorHandler.js";

const app = express();
const PORT = process.env.PORT ?? 3000;

app.use(cors({ origin: "http://localhost:5173" }));
app.use(express.json());

app.get("/api/health", (req, res) => {
    res.json({ status: "ok" });
});

app.use("/api/products", productRoutes);
```

```
app.use(notFound);
app.use(errorHandler);

app.listen(PORT, () => {
    console.log(`Server pornit pe http://localhost:${PORT}`);
});
```

Teste:

```
curl http://localhost:3000/api/products

curl http://localhost:3000/api/products/1

curl -X POST http://localhost:3000/api/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Monitor","price":900,"category":"Electronics","stock":8}'
```

> Momentan oricine poate crea si sterge produse. In sectiunile 6-7 adaugam autentificarea si protejam aceste rute.

6. Autentificare cu JWT

Autentificare vs autorizare

Concept	Intrebare	Exemplu
Autentificare	Cine esti?	login cu email si parola
Autorizare	Ce ai voie sa faci?	doar ADMIN poate sterge produse

Parolele

Parolele **nu se salveaza niciodata in text clar**. Se salveaza un **hash**:

```
"parola123"  →  bcrypt  →  "$2a$10$N9qo8uLOickgx2ZMRZoMye..."
```

- hash-ul nu poate fi transformat inapoi in parola;
- bcrypt adauga automat un *salt*, deci doua parole identice produc hash-uri diferite;
- la login comparam parola introdusa cu hash-ul din baza de date.

```
const hash = await bcrypt.hash("parola123", 10);   // 10 = cost factor

const ok = await bcrypt.compare("parola123", hash); // true
```

JWT (JSON Web Token)

Un JWT este un sir de caractere format din 3 parti:

```
header.payload.signature
```

Exemplu de payload:

```
{
    "userId": 5,
    "role": "USER",
    "iat": 1767000000,
    "exp": 1767003600
}
```

- **payload** contine datele utilizatorului si data expirarii (`exp`);
- **signature** este generata cu `JWT_SECRET`. Daca payload-ul este modificat, semnatura nu mai este valida.

> Payload-ul este doar codat (Base64), **nu criptat**. Orice persoana il poate citi. Nu punem in el parole sau date sensibile.

Unde pastram token-ul in browser?

Dupa login, token-ul trebuie sa ajunga la client si sa fie trimis la fiecare request. Exista doua abordari frecvente:

	Token in raspunsul JSON (+ `localStorage`)	**Cookie HttpOnly**
Poate fi citit din JavaScript	da	**nu**
Poate fi furat de un script injectat (XSS)	da, direct din `localStorage`	nu poate fi citit sau copiat
Cum ajunge la server	manual, prin header-ul `Authorization`	**automat**, de catre browser
Risc CSRF	nu	da, dar se reduce cu `SameSite`
Logout	clientul sterge token-ul	serverul sterge cookie-ul

In acest seminar folosim **cookie HttpOnly**. Serverul trimite token-ul in header-ul `Set-Cookie`, **nu** in corpul raspunsului. Browserul il salveaza si il ataseaza singur la fiecare request catre API, iar codul JavaScript din React nu il vede niciodata.

> HttpOnly nu elimina complet riscul XSS: un script injectat poate inca face request-uri in numele utilizatorului, cat timp pagina este deschisa. Ce nu mai poate face este sa **fure token-ul** si sa il foloseasca de pe alt dispozitiv.

Atributele cookie-ului

Atribut	**Valoare**	**Rol**
`httpOnly`	`true`	cookie-ul nu poate fi citit din JavaScript (`document.cookie`)
`secure`	`true` in productie	cookie-ul se trimite doar prin HTTPS. In dezvoltare (`http://localhost`) ramane `false`
`sameSite`	`"lax"`	browserul nu trimite cookie-ul la request-uri initiate de alte site-uri (protectie CSRF)
`maxAge`	`3600000` (1 ora)	durata de viata, in milisecunde. Trebuie sa corespunda cu `JWT_EXPIRES_IN`
`path`	`"/"`	cookie-ul se trimite la toate rutele

> `localhost:5173` (React) si `localhost:3000` (API) sunt considerate **acelasi site** (portul nu conteaza pentru `SameSite`), deci `sameSite: "lax"` functioneaza in dezvoltare. Daca in productie frontend-ul si API-ul sunt pe domenii diferite, ar fi necesar `sameSite: "none"` impreuna cu `secure: true`.

Fluxul de autentificare

```
Client                                Server
  |                                     |
  | POST /api/auth/login                |
  | { email, password }  ---------->    |
  |                                     | verifica parola (bcrypt)
  |                                     | genereaza JWT
  | <----------------  { user }         |
  |     Set-Cookie: token=<JWT>;        |
  |     HttpOnly; SameSite=Lax          |
  |                                     |
  | GET /api/orders                     |
  | Cookie: token=<JWT>  ---------->    | (atasat automat de browser)
  |                                     | verifica token-ul
  |                                     | → identifica utilizatorul
  | <----------  datele comenzilor      |
```

Serverul **nu pastreaza sesiuni**. Fiecare request contine cookie-ul cu token-ul, iar serverul il verifica.

Configurarea cookie-urilor

Mai intai activam citirea cookie-urilor si permitem trimiterea lor intre `localhost:5173` si `localhost:3000`. In `index.js`:

```
import cookieParser from "cookie-parser";

app.use(cors({
    origin: "http://localhost:5173",
    credentials: true
}));
app.use(express.json());
app.use(cookieParser());
```

- `cookieParser()` populeaza `req.cookies`, la fel cum `express.json()` populeaza `req.body`;
- `credentials: true` permite browserului sa accepte si sa trimita cookie-uri la request-uri cross-origin;
- cu `credentials: true`, `origin` **nu poate fi** `"*"`, trebuie sa fie adresa exacta a frontend-ului.

Apoi centralizam setarile intr-un fisier separat, pentru ca aceleasi optiuni trebuie folosite la creare si la stergere.

Fisierul `src/utils/cookies.js`:

```
export const AUTH_COOKIE = "token";

const ONE_HOUR = 60 * 60 * 1000; // trebuie sa corespunda cu JWT_EXPIRES_IN
```

```
const cookieOptions = {
    httpOnly: true,
    secure: process.env.NODE_ENV === "production",
    sameSite: "lax",
    path: "/"
};

export const setAuthCookie = (res, token) => {
    res.cookie(AUTH_COOKIE, token, { ...cookieOptions, maxAge: ONE_HOUR });
}

export const clearAuthCookie = (res) => {
    res.clearCookie(AUTH_COOKIE, cookieOptions);
}
```

> `clearCookie` sterge cookie-ul doar daca `path`, `sameSite` etc. sunt aceleasi ca la creare. De aceea folosim acelasi obiect `cookieOptions`.

Rutele `register` si `login`

Fisierul `src/routes/auth.routes.js`:

```
import { Router } from "express";
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import { prisma } from "../lib/prisma.js";
import { HttpError } from "../utils/httpError.js";
import { setAuthCookie, clearAuthCookie } from "../utils/cookies.js";

const router = Router();

export const signToken = (signingdata) => {
    return jwt.sign(
        signingdata,
        process.env.JWT_SECRET,
        { expiresIn: process.env.JWT_EXPIRES_IN ?? "1h" }
    );
}

// scoatem parola din obiect inainte de a-l trimite clientului
const toPublicUser = ({ password, ...user }) => {
    return user;
}

// POST /api/auth/register
router.post("/register", async (req, res) => {
    const { name, email, password } = req.body;

    const invalid = [name, email, password].some(
        value => typeof value !== "string" || value.trim() === ""
    );
```

```
    if (invalid) {
        throw new HttpError(400, "name, email si password sunt obligatorii");
    }

    if (!/^\S+@\S+\.\S+$/.test(email)) {
        throw new HttpError(400, "Email invalid");
    }

    if (password.length < 6) {
        throw new HttpError(400, "Parola trebuie sa aiba minim 6 caractere");
    }

    const normalizedEmail = email.trim().toLowerCase();

    const existing = await prisma.user.findUnique({
        where: { email: normalizedEmail }
    });

    if (existing) {
        throw new HttpError(409, "Exista deja un cont cu acest email");
    }

    const user = await prisma.user.create({
        data: {
            name: name.trim(),
            email: normalizedEmail,
            password: await bcrypt.hash(password, 10)
        }
    });

    res.status(201).send();
});

// POST /api/auth/login
router.post("/login", async (req, res) => {
    const { email, password } = req.body;

    if (typeof email !== "string" || typeof password !== "string") {
        throw new HttpError(400, "email si password sunt obligatorii");
    }

    const user = await prisma.user.findUnique({
        where: { email: email.trim().toLowerCase() }
    });

    const passwordOk = user
        ? await bcrypt.compare(password, user.password)
        : false;

    if (!passwordOk) {
        // acelasi mesaj pentru ambele cazuri, ca sa nu aflam ce emailuri exista
        throw new HttpError(401, "Email sau parola incorecta");
    }
```

```
    const publicUser = toPublicUser(user);

    setAuthCookie(res, signToken(publicUser));

    res.json({ user: toPublicUser(publicUser) });
});

// POST /api/auth/logout
router.post("/logout", (req, res) => {
    clearAuthCookie(res);
    res.status(204).send();
});

export default router;
```

Observatii:

- utilizatorul nou primeste implicit `role = "USER"`. Campul `role` **nu se citeste din** `req.body`, altfel oricine si-ar putea da drepturi de admin;
- mesajul de la login este identic pentru email inexistent si parola gresita;
- raspunsul **nu mai contine token-ul**. Acesta este trimis doar in header-ul `Set-Cookie`, deci codul JavaScript din frontend nu il poate citi;
- `POST /api/auth/logout` este necesara pentru ca un cookie HttpOnly **nu poate fi sters din JavaScript**. Doar serverul il poate elimina.

Montare in `index.js`:

```
import authRoutes from "./routes/auth.routes.js";

app.use("/api/auth", authRoutes);
```

Test:

```
curl -i -X POST http://localhost:3000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Ana","email":"ana@test.com","password":"parola123"}'
```

Optiunea `-i` afiseaza header-ele raspunsului. Verificati ca:

- in header-e apare `Set-Cookie: token=...; HttpOnly; SameSite=Lax`;
- in corpul JSON exista doar `user`, **fara** `token`.

7. Autorizare: middleware si roluri

Middleware `authenticate`

Citeste token-ul din cookie-ul `token`, il verifica si salveaza utilizatorul in `req.user`.

Fisierul `src/middleware/auth.js`:

```
import jwt from "jsonwebtoken";
import { HttpError } from "../utils/httpError.js";
import { AUTH_COOKIE } from "../utils/cookies.js";

export const authenticate = (req, res, next) => {
    const token = req.cookies?.[AUTH_COOKIE];

    if (!token) {
        throw new HttpError(401, "Lipseste token-ul de autentificare");
    }

    let payload;

    try {
        payload = jwt.verify(token, process.env.JWT_SECRET);
    } catch {
        throw new HttpError(401, "Token invalid sau expirat");
    }

    req.user = { id: payload.userId, role: payload.role };

    next();
}

export const requireRole = (role) => {
    return (req, res, next) => {
        if (req.user?.role !== role) {
            throw new HttpError(403, "Nu ai dreptul sa accesezi aceasta resursa");
        }

        next();
    };
}
```

Observati:

- `req.cookies?.[AUTH_COOKIE]` foloseste optional chaining (Seminar 1, sectiunea 7). Fara `cookieParser()`, `req.cookies` ar fi `undefined`;
- `requireRole` este o functie care **returneaza** un middleware;
- `requireRole` se foloseste **dupa** `authenticate`, pentru ca se bazeaza pe `req.user`.

Protejarea rutelor

Middleware-urile se pun inaintea handler-ului:

```
import { authenticate, requireRole } from "../middleware/auth.js";

router.post("/", authenticate, requireRole("ADMIN"), async (req, res) => { ... });

router.patch("/:id", authenticate, requireRole("ADMIN"), async (req, res) => { ...
});

router.delete("/:id", authenticate, requireRole("ADMIN"), async (req, res) => {
... });
```

Rutele `GET /api/products` si `GET /api/products/:id` raman publice.

Ruta `GET /api/auth/me`

Returneaza utilizatorul curent. Frontend-ul o foloseste pentru pagina de profil. Se adauga in `auth.routes.js`:

```
import { authenticate } from "../middleware/auth.js";

// GET /api/auth/me
router.get("/me", authenticate, async (req, res) => {
    const user = await prisma.user.findUnique({
        where: { id: req.user.id }
    });

    if (!user) {
        throw new HttpError(401, "Utilizatorul nu mai exista");
    }

    res.json(toPublicUser(user));
});
```

Test complet

```
# 1. login ca admin (creat prin seed)
#    -c salveaza cookie-ul primit in fisierul cookies.txt
curl -c cookies.txt -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@shop.com","password":"admin123"}'

# 2. -b trimite cookie-ul salvat (browserul face asta automat)
curl -b cookies.txt -X POST http://localhost:3000/api/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Monitor","price":900,"category":"Electronics","stock":8}'
```

```
# 3. logout: serverul sterge cookie-ul
curl -b cookies.txt -c cookies.txt -X POST http://localhost:3000/api/auth/logout
```

Rezultate asteptate:

Situatie	Status
fara cookie (nelogat)	`401`
cookie modificat sau expirat	`401`
cookie de utilizator obisnuit (`USER`)	`403`
cookie de `ADMIN`	`201`
dupa `POST /api/auth/logout`	`401`

In Postman, cookie-ul primit la login este salvat si retrimis automat (vezi butonul **Cookies** de sub bara de URL). Nu mai este nevoie de tab-ul **Authorization**.

> **Pentru Seminar 3:** React trebuie sa trimita cookie-urile explicit. Cu `fetch` se adauga `credentials: "include"` la fiecare request, iar cu axios `withCredentials: true`. Fara aceasta optiune, browserul nu trimite cookie-ul catre API.

8. Relatii si comenzi

Un magazin online are nevoie de **comenzi**. Ele leaga tabelele intre ele.

Modelul de date

```
User  1 ──────── N  Order  1 ──────── N  OrderLine  N ──────── 1  Product
```

- un utilizator poate avea mai multe comenzi (**one-to-many**);
- o comanda contine mai multe linii (`OrderLine`);
- fiecare linie se refera la un produs;
- `Order` si `Product` sunt astfel intr-o relatie **many-to-many**, realizata prin tabela intermediara `OrderLine`, care mai retine `quantity` si `unitPrice`.

Schema

Adaugam modelele noi in `schema.prisma`:

```
model Order {
    id        Int         @id @default(autoincrement())
    status    String      @default("PENDING")
    total     Float
    createdAt DateTime    @default(now())

    userId    Int
    user      User        @relation(fields: [userId], references: [id])
    orderLines     OrderLine[]
}

model OrderLine {
    id        Int     @id @default(autoincrement())
    quantity  Int
    unitPrice Float

    orderId   Int
    productId Int
    order     Order   @relation(fields: [orderId], references: [id], onDelete:
Cascade)
    product   Product @relation(fields: [productId], references: [id])
}
```

Relatiile trebuie declarate si pe **partea cealalta**. Modificam modelele existente:

```
model User {
    // ... campurile existente ...
    orders Order[]
}

model Product {
    // ... campurile existente ...
    orderLine OrderLine[]
}
```

Explicatii:

Element	Rol
`userId Int`	**cheia straina** (foreign key), stocata in baza de date
`user User @relation(...)`	camp virtual, permite accesarea obiectului `User`
`fields: [userId]`	coloana din tabela curenta
`references: [id]`	coloana referita din cealalta tabela
`Order[]`	partea "many" a relatiei
`onDelete: Cascade`	daca stergem comanda, se sterg si liniile ei

Cheia straina se afla intotdeauna pe partea **"many"**.

`unitPrice` salveaza pretul din momentul comenzii. Daca produsul se scumpeste ulterior, comenzile vechi nu se modifica.

Migrare:

```
npx prisma migrate dev --name add_orders
```

`include` si `select`

Implicit, Prisma nu aduce datele din relatii. Le cerem explicit:

```
const orders = await prisma.order.findMany({
    include: {
        orderLines: {
            include: { product: true }
        }
    }
});
```

Rezultat:

```
{
    "id": 1,
    "status": "PENDING",
    "total": 6500,
    "userId": 2,
    "orderLines": [
        {
            "id": 1,
            "quantity": 1,
            "unitPrice": 4000,
            "product": { "id": 1, "name": "Laptop" }
        }
    ]
}
```

`select` in interiorul unui `include` limiteaza campurile:

```
include: {
    orderLines: {
        include: {
            product: { select: { id: true, name: true, imageUrl: true } }
        }
    }
}
```

Creare cu relatii (nested create)

```
await prisma.order.create({
    data: {
        userId: 2,
        total: 6500,
        orderLines: {
            create: [
                { productId: 1, quantity: 1, unitPrice: 4000 },
                { productId: 2, quantity: 1, unitPrice: 2500 }
            ]
```

```
        }
    }
});
```

Comanda si liniile ei se creeaza dintr-o singura operatie.

Tranzactii

La plasarea unei comenzi se intampla mai multe lucruri:

```
1. verificam stocul
2. scadem stocul produselor
3. cream comanda si liniile ei
```

Daca pasul 3 esueaza dupa pasul 2, am pierdut stoc fara sa existe comanda. De aceea folosim o **tranzactie**: ori reusesc toate operatiile, ori niciuna.

```
await prisma.$transaction(async (tx) => {
    // folosim "tx" in loc de "prisma"
    // daca aruncam o eroare, toate modificarile sunt anulate (rollback)
});
```

Ruta `POST /api/orders`

Frontend-ul trimite doar produsele si cantitatile:

```
{
    "orderLines": [
        { "productId": 1, "quantity": 1 },
        { "productId": 2, "quantity": 2 }
    ]
}
```

> **Preturile nu se primesc de la client.** Orice utilizator ar putea trimite `price: 0`. Serverul citeste preturile din baza de date si calculeaza singur totalul.

Fisierul `src/routes/orders.routes.js`:

```
import { Router } from "express";
import { prisma } from "../lib/prisma.js";
import { authenticate } from "../middleware/auth.js";
import { HttpError } from "../utils/httpError.js";

const router = Router();
```

```
const orderInclude = {
    orderLines: {
        include: {
            product: { select: { id: true, name: true, imageUrl: true } }
        }
    }
};

// POST /api/orders
router.post("/", authenticate, async (req, res) => {
    const { orderLines } = req.body;

    if (!Array.isArray(orderLines) || orderLines.length === 0) {
        throw new HttpError(400, "Comanda trebuie sa contina cel putin un
produs");
    }

    // unificam produsele duplicate: productId -> cantitate totala
    const quantities = new Map();

    for (const item of orderLines) {
        const { productId, quantity } = item ?? {};

        if (!Number.isInteger(productId) || !Number.isInteger(quantity) ||
quantity < 1) {
            throw new HttpError(
                400,
                "productId si quantity trebuie sa fie numere intregi pozitive"
            );
        }

        quantities.set(productId, (quantities.get(productId) ?? 0) + quantity);
    }

    const order = await prisma.$transaction(async (tx) => {

        const products = await tx.product.findMany({
            where: { id: { in: [...quantities.keys()] } }
        });

        if (products.length !== quantities.size) {
            throw new HttpError(404, "Unul dintre produse nu exista");
        }

        let total = 0;
        const orderLine = [];

        for (const product of products) {
            const quantity = quantities.get(product.id);

            // scadem stocul doar daca exista suficient
            const { count } = await tx.product.updateMany({
                where: { id: product.id, stock: { gte: quantity } },
```

```
                data: { stock: { decrement: quantity } }
            });

            if (count === 0) {
                throw new HttpError(409, `Stoc insuficient pentru
"${product.name}"`);
            }

            total += product.price * quantity;

            orderLine.push({
                productId: product.id,
                quantity,
                unitPrice: product.price
            });
        }

        return tx.order.create({
            data: {
                userId: req.user.id,
                total: Math.round(total * 100) / 100,
                orderLines: { create: orderLine }
            },
            include: orderInclude
        });
    });

    res.status(201).json(order);
});

// GET /api/orders  (comenzile utilizatorului curent)
router.get("/", authenticate, async (req, res) => {
    const orders = await prisma.order.findMany({
        where: { userId: req.user.id },
        include: orderInclude,
        orderBy: { createdAt: "desc" }
    });

    res.json(orders);
});

export default router;
```

Observatii:

- `updateMany` cu conditia `stock: { gte: quantity }` verifica si scade stocul intr-o singura operatie, deci doua comenzi simultane nu pot lua acelasi stoc de doua ori;
- `throw new HttpError(...)` in interiorul tranzactiei anuleaza automat toate modificarile facute pana atunci;
- `userId` vine din token (`req.user.id`), nu din body. Un utilizator nu poate crea comenzi in numele altcuiva;
- `GET /api/orders` filtreaza dupa `userId`, deci fiecare utilizator isi vede doar comenzile.

Montare in `index.js`:

```
import orderRoutes from "./routes/orders.routes.js";

app.use("/api/orders", orderRoutes);
```

9. Structura finala si checklist

Rutele API-ului

Metoda	Ruta	Acces
POST	/api/auth/register	public
POST	/api/auth/login	public
POST	/api/auth/logout	public
GET	/api/auth/me	utilizator autentificat
GET	/api/products	public
GET	/api/products/:id	public
POST	/api/products	ADMIN
PATCH	/api/products/:id	ADMIN
DELETE	/api/products/:id	ADMIN
POST	/api/orders	utilizator autentificat
GET	/api/orders	utilizator autentificat

Fisierul `index.js` final

```
import "dotenv/config";
import express from "express";
import cors from "cors";
import cookieParser from "cookie-parser";

import authRoutes from "./routes/auth.routes.js";
import productRoutes from "./routes/products.routes.js";
import orderRoutes from "./routes/orders.routes.js";
import { notFound, errorHandler } from "./middleware/errorHandler.js";

const app = express();
const PORT = process.env.PORT ?? 3000;

app.use(cors({
    origin: "http://localhost:5173",
```

```
    credentials: true
}));
app.use(express.json());
app.use(cookieParser());

app.get("/api/health", (req, res) => {
    res.json({ status: "ok" });
});

app.use("/api/auth", authRoutes);
app.use("/api/products", productRoutes);
app.use("/api/orders", orderRoutes);

app.use(notFound);
app.use(errorHandler);

app.listen(PORT, () => {
    console.log(`Server pornit pe http://localhost:${PORT}`);
});
```

Checklist

Inainte de Seminar 3, verificati ca:

- [ ] `GET /api/products` returneaza produsele din seed;
- [ ] `POST /api/auth/register` creeaza un utilizator si seteaza cookie-ul `token` (HttpOnly); raspunsul **nu** contine token-ul;
- [ ] `POST /api/auth/login` functioneaza si cu `admin@shop.com` / `admin123`;
- [ ] in browser (DevTools → Application → Cookies) cookie-ul `token` are bifat **HttpOnly**, iar `document.cookie` din consola nu il afiseaza;
- [ ] `POST /api/auth/logout` sterge cookie-ul, iar ruta `GET /api/auth/me` raspunde apoi `401`;
- [ ] `POST /api/products` raspunde `401` fara cookie si `403` cu cookie de `USER`;
- [ ] `POST /api/orders` creeaza o comanda si scade stocul in baza de date (verificati in Prisma Studio);
- [ ] o comanda cu cantitate mai mare decat stocul returneaza `409`, iar stocul ramane neschimbat;
- [ ] `GET /api/orders` returneaza doar comenzile utilizatorului curent;
- [ ] parola **nu** apare in niciun raspuns.

Greseli frecvente

Problema	Cauza probabila
`Cannot use import statement outside a module`	lipseste `"type": "module"` din `package.json`
`req.body` este `undefined`	lipseste `app.use(express.json())` sau header-ul `Content-Type: application/json`
`secretOrPrivateKey must have a value`	`JWT_SECRET` lipseste din `.env` sau `dotenv` nu este importat primul
`Unknown argument` la Prisma	schema a fost modificata fara `migrate dev`

Problema	Cauza probabila
Eroare CORS in browser	`origin` din `cors(...)` nu corespunde cu adresa aplicatiei React
`req.cookies` este `undefined`	lipseste `app.use(cookieParser())` sau este declarat dupa rute
Cookie-ul apare la login, dar nu se trimite la request-urile urmatoare	lipseste `credentials: "include"` (fetch) / `withCredentials: true` (axios) in client
Eroare CORS "Access-Control-Allow-Credentials"	lipseste `credentials: true` in `cors(...)` sau `origin` este `"*"`
`401` dupa logout, dar cookie-ul ramane in browser	`clearCookie` a fost apelat cu alte optiuni (`path`, `sameSite`) decat `res.cookie`
Importuri locale care nu merg	in ES modules extensia `.js` este obligatorie: `import x from "./file.js"`